COMS W4156 Advanced Software Engineering (ASE)

October 26, 2023

Agenda

1. First Assessment posts at 12:01am tonight
2. CRC Design

Assignments

- ➔ First Iteration past due
- ➔ First Iteration Demo due tonight by 11:59pm

First Assessment posts tonight 12:01am Friday October 27 through 11:59pm Monday October 30

- ➔ Second Iteration due Thursday, November 30, by 11:59pm
- ➔ Second Iteration Demo due Thursday, December 7, by 11:59pm

How Long Will The Assessment Take?

You will have four days (minus two minutes) to complete the assessment

The assessment will not be timed and your time spent need not be consecutive

You can spend as much or as little time as you want within the four days

The assessment is expected to take 4-6 hours total

If you spend significantly less than 4 hours, something may be wrong

If a question seems "too easy", something may be wrong

If your answer seems "too short", something may be wrong

Is The Assessment Open Book?

You can read anything anywhere you normally have access to

You cannot post questions about the assessment anywhere except edstem

You cannot ask anyone questions except the teaching staff

You cannot discuss the assessment with anyone except the teaching staff

Ask More Questions NOW

Mandatory Ungraded Question

You have 100 total points to allocate among the members of your team. **Assign some number of points (possibly 0) to each member of your team based on what you believe was that student's relative participation in the team project thus far, and *explain your scores*.** Make sure the total is exactly 100. Remember to include yourself! For allocating points, consider level of effort and contributions for anything relevant to the team project. Note equal participation is not the same thing as equal lines of code produced. Please do not predict the future, consider only the contributions to the team project as of the date of writing. It is very important that you explain the scores you give to each student, including yourself: *If you do not provide meaningful explanations, then your response will be considered incomplete and you will receive 0 overall on the assessment.*

For example, in a team where all team members participated more or less equally, the scores would be: Cora 25, Kaedi 25, Kaylah 25, Nina 25, and your explanation would describe who did which part(s) of the team project effort thus far.

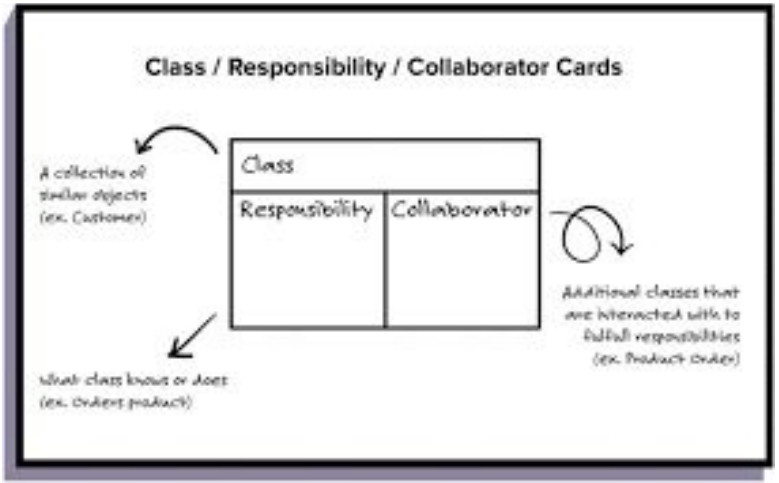
In a team with a substantial disparity in contributions, however, the scores might be: Hawkeye 35, Captain Marvel 35, Hulk 25, Thor 5, and your explanation might describe Hawkeye and Captain Marvel as doing most of the work, and Hulk helped, but Thor rarely showed up for team meetings and did not seem interested in the project. Again, your explanation should describe who did what, or who did not do what, as the case may be.

Optional Ungraded Question

Is there anything that you think should have been covered already but wasn't? Is there anything that was covered unnecessarily or should have been put off until later? Is there anything that should have been presented in a different way? Is there anything else you would change about the course? Please explain. Suggestions for the remainder of this course will be greatly appreciated. In addition to submitting comments and suggestions here as part of your first assessment, you can also post anonymously on edstem. (And please also submit the SEAS evaluation.)

Agenda

1. First Assessment posts at 12:01am tonight
2. CRC Design



How do you go from Use Cases to Code?

Start coding

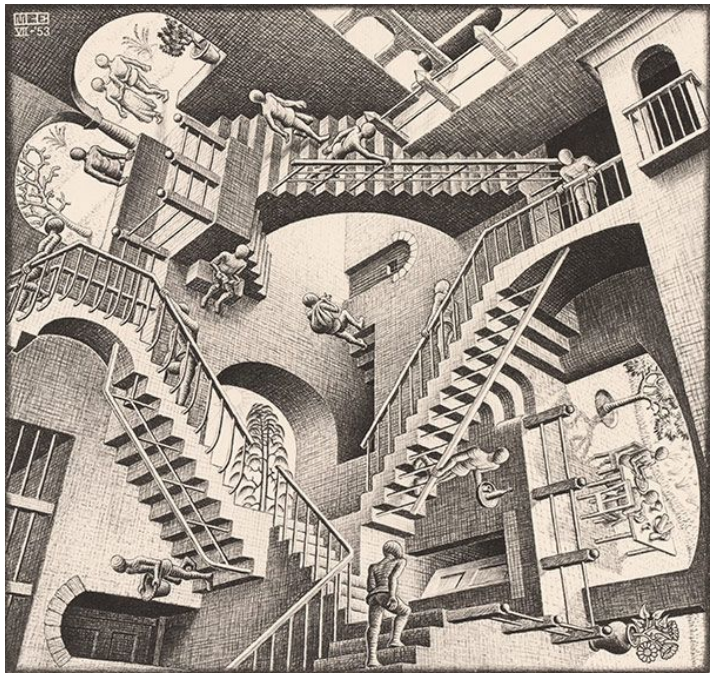
Outsource (Buy vs Build)

Design (and Architecture)

In "agile" software development, use cases and/or user stories should only capture features intended for the current iteration (or, sometimes, the upcoming release)

If there is too much to do, prioritize and push off lower priority to later

Example design: good art, bad code



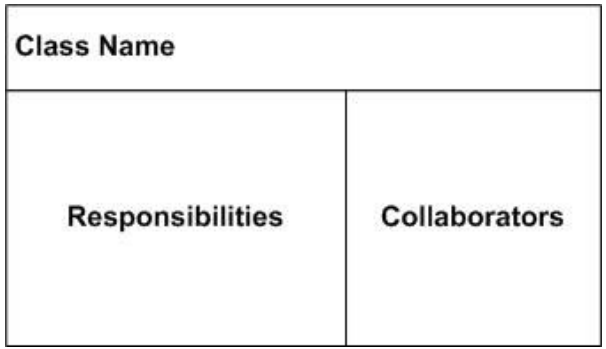
Agile Design: CRC = Class Responsibilities Collaborators

Like user stories, CRC can be written on index cards, so often known as “CRC cards”

Each *class* is a thing, entity or object

Each *responsibility* is some task the entity needs to do or data it needs to track

Collaborators are other classes the entity interacts with



CRC focuses on the *purpose* of each entity rather than its processes, data flows and data stores (procedural design)

Find Classes

Start with the set of use cases and/or user stories

Find the *nouns* other than the actor (or role) - classes should be the entities that “do” actions, not the actors who initiate actions

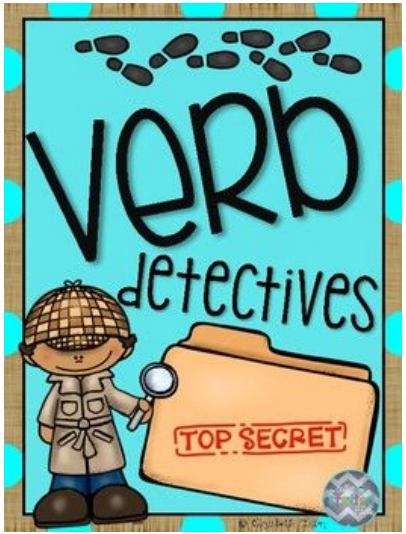
Convert plurals to singular, merge synonyms

Beware adjectives - may mean a whole new class,
e.g., “car” vs. “toy car”

Beware hidden nouns, e.g., passive voice
“the thing is activated” = “SOMETHING activates the thing”

Some nouns may be attributes of entities, not themselves entities, e.g., “the radius of a circle” - here the circle is an entity and the radius is an attribute of the circle, it has no meaning as a standalone entity

Find Responsibilities



Find the *verbs* (some may be CRUD)

Say WHAT gets done, not HOW it gets done

Turn passive verbs into active verbs
“the thing is activated” = “SOMETHING activates the thing”

The classes and responsibilities create a vocabulary for discussing the design

Find Collaborators

Find any relationships or associations among entities - not necessarily symmetric, can be one-way

Any entity that a class needs to know about, communicate with, contain, control, or that otherwise help it perform one or more responsibilities that it cannot perform on its own

If that entity does not already have its own CRC card, make a new card

If that entity does not already have a responsibility to do something or provide something on behalf of the class at hand, add that responsibility (and any additional collaborators it needs)

CRC Design Process

Develop a set of CRC cards for the current set of user stories / use cases

CRC does not need an object-oriented programming language - CRC can stand for *Component* Responsibilities Collaborators

Walk through the steps and flows of
each use case and *animate* the CRC cards:
move index cards on a table or sticky notes on a board

Make sure everything is a responsibility of some class,
and figure out what each class needs from collaborators
to fulfill its responsibilities

There is no Global Master

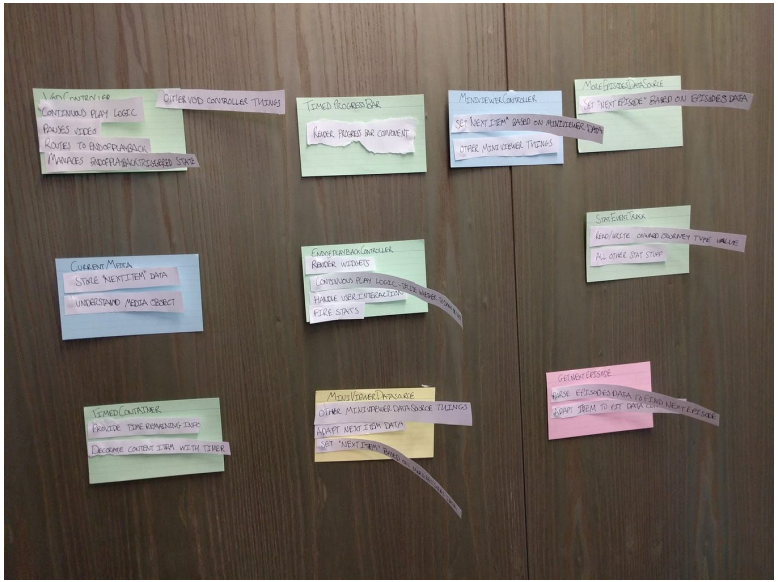
When “the system” does something, that means some class in the system does it

There should not be any global master that does everything and/or is related to everything else

Give up global knowledge of control and rely on local knowledge of objects to accomplish their tasks

Debug the CRC cards

- Remember classes are nouns that “do” operations, not the actors who initiate operations
- Split classes with too many responsibilities
- Combine classes with too few responsibilities
- Remove redundancy



[CRC Card Maker]

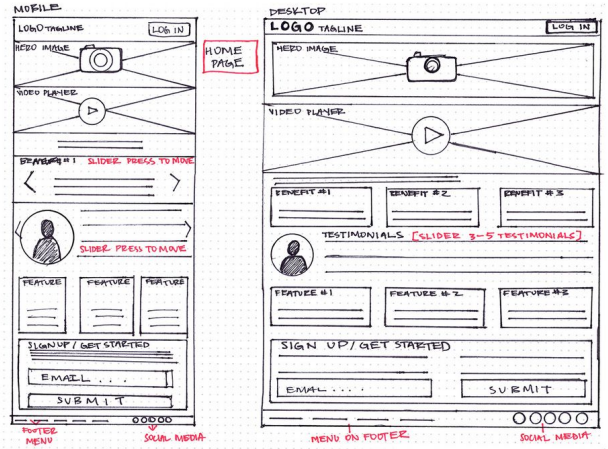
NOW Start Coding

Class -> class, module, file

Responsibility -> method, function, subroutine

Collaborator -> calls, parameters, shared data

GUI applications:
wireframes might or might not be developed first,
then start coding



Example User Stories -> CRC?

As a PC user, I want to backup my entire hard drive so that my files are safe

As a PC user, I want to specify which folders to backup to make sure my most important files are saved

As a PC user, I want to specify subfolders and filetypes to skip within the backup folders so my backup store isn't filled with things I don't need

Example Turning User Stories into Software

What other information might the developer need to start coding? - Another way to think of it is what additional information will the backup program need?

What permissions will the backup program need to access the files its supposed to backup

Where to put the backup copies of the files

What to do if the backup store is not available

What to do if the backup store is full

What to do with the files already in the backup store

Etc…

Example Use Case -> CRC?

1. On Wednesdays, the housekeeper reports to the laundry room.
2. The housekeeper sorts the laundry that is there.
3. Then they wash each load.
4. They dry each load.
5. A. They fold the items that need folding. ⇐ basic flow

   B. They iron and hang the items that are wrinkled. ⇐ alternative flow

   C. They throw away any laundry item that is irrevocably shrunken, soiled or scorched. ⇐ alternative flow

Actors: Housekeeper, system that transports laundry to the laundry room.

Stakeholders: Owners/wearers of the laundry items.

Trigger: Dirty laundry has been transported to the laundry room.

Precondition: It is Wednesday.

Postcondition: Clean laundry is folded or hung. Damaged laundry is discarded.

Example - Further Details and Possible Exception Flows

1. A. The housekeeper reports to the laundry room.
   B. The housekeeper does not show up. ⇐ **exception flow**
   C. The laundry room is flooded (or other reasons the housekeeper cannot get to the laundry room). ⇐ **exception flow**
2. A. The housekeeper sorts the laundry that is there into hot, warm, cold wash (or other categories). ⇐ more detail needed
   B. The laundry is missing. ⇐ **exception flow**
3. A. Then they wash each load (in what order? what detergent, etc. is placed in washer?). ⇐ more detail needed
   B. The washing machine does not work. ⇐ **exception flow**
   C. The xxx load is too large for the washing machine and needs to be split (etc). ⇐ more detail needed / **exception**
   D. Washing machine takes too long to do all the loads before the housekeeper has to leave. ⇐ **exception flow**
4. And so on….

Example CRC Cards

ATM

Doctors Office

Drawing program

Assignments

- ➔ First Iteration past due
- ➔ First Iteration Demo due tonight by 11:59pm

First Assessment posts tonight 12:01am Friday October 27 through 11:59pm Monday October 30

- ➔ Second Iteration due Thursday, November 30, by 11:59pm
- ➔ Second Iteration Demo due Thursday, December 7, by 11:59pm

Next Week

Discuss Second Iteration

Design Principles

Ask Me Anything